EXPERIMENT 10: AJAX-Based Web Application

Aim: To implement AJAX applications that enable asynchronous communication with servers without reloading the webpage.

STEPS:-

Step 1: Creation of Project Structure

A project folder was created containing the required files such as *index.html*, *style.css*, and *script.js*. These files were used to structure the webpage, apply styling, and implement functionality respectively.

Step 2: Designing the User Interface

A basic HTML page was developed with a heading, a search input field, a button to trigger data fetching, and a container to display the fetched posts. The layout was styled using CSS to make the interface clean and user-friendly.

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
  <title>Exp 10 - AJAX Web Application</title>
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; }

    body {
      font-family: Arial, sans-serif;
      background: #f0f2f5;
      padding: 30px;
    }

    h1 {
      text-align: center;
      margin-bottom: 20px;
      color: #333;
    }

    .controls {
      display: flex;
```

```
  justify-content: center;
  gap: 12px;
  margin-bottom: 24px;
  flex-wrap: wrap;
}

button {
  padding: 10px 22px;
  border: none;
  border-radius: 6px;
  cursor: pointer;
  font-size: 14px;
  background: #4f46e5;
  color: #fff;
  transition: background 0.2s;
}

button:hover { background: #4338ca; }
button:disabled { background: #a5b4fc; cursor: not-allowed; }

#status {
  text-align: center;
  font-size: 13px;
  color: #666;
  margin-bottom: 16px;
  min-height: 18px;
}

#output {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(280px, 1fr));
  gap: 16px;
}

.card {
  background: #fff;
  border-radius: 8px;
  padding: 16px;
  box-shadow: 0 1px 4px rgba(0,0,0,0.1);
}

.card h3 {
  font-size: 15px;
  color: #1e1b4b;
  margin-bottom: 8px;
  text-transform: capitalize;
}

.card p {
  font-size: 13px;
  color: #555;
  line-height: 1.5;
}
```

```
    .card .meta {
      margin-top: 10px;
      font-size: 12px;
      color: #999;
    }

    .error {
      text-align: center;
      color: #dc2626;
      font-size: 14px;
      grid-column: 1 / -1;
    }
  </style>
</head>
<body>

  <h1>AJAX Web Application</h1>

  <div class="controls">
    <button id="btnPosts">Fetch Posts</button>
    <button id="btnUsers">Fetch Users</button>
    <button id="btnClear">Clear</button>
  </div>

  <div id="status"></div>
  <div id="output"></div>

  <script src="script.js"></script>
</body>
</html>
```

Step 3: Implementing AJAX Request

JavaScript was used to implement AJAX functionality using the XMLHttpRequest object. A GET request was sent to the Spaceflight News API to retrieve article data asynchronously without reloading the webpage.

script.js

```
const output = document.getElementById("output");
const status = document.getElementById("status");
const btnPosts = document.getElementById("btnPosts");
const btnUsers = document.getElementById("btnUsers");
const btnClear = document.getElementById("btnClear");

// Generic AJAX fetch using XMLHttpRequest
function ajaxGet(url, onSuccess, onError) {
  const xhr = new XMLHttpRequest();

  // Step 2: Open a GET request
  xhr.open("GET", url, true); // true = asynchronous
```

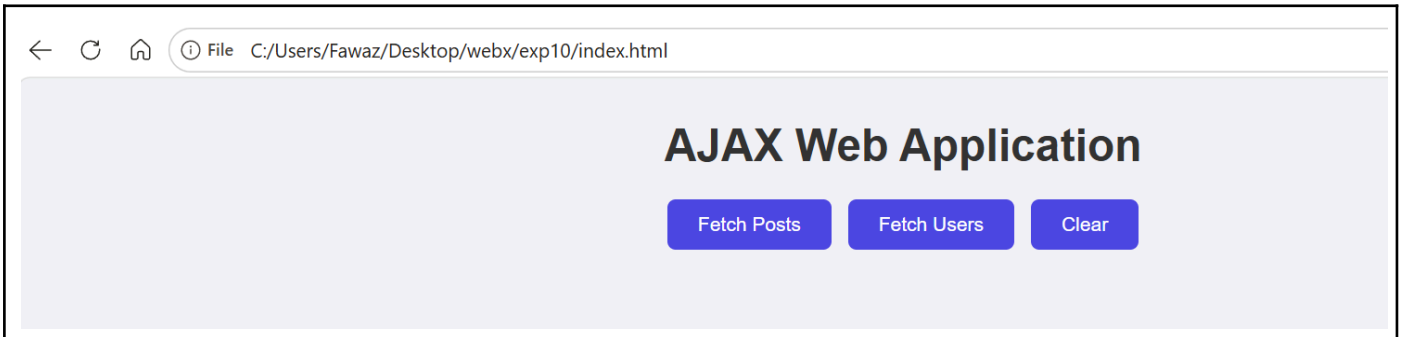
```
  // Step 3: Monitor request state changes
  xhr.onreadystatechange = function () {
    if (xhr.readyState === XMLHttpRequest.DONE) {
      if (xhr.status === 200) {
        // Convert JSON string to object
        const data = JSON.parse(xhr.responseText);
        onSuccess(data);
      } else {
        onError(`Request failed with status: ${xhr.status}`);
      }
    }
  };

  xhr.onerror = () => onError("Network error occurred.");
  xhr.send();
}

// Step 4: Render posts into cards
function renderPosts(posts) {
  output.innerHTML = posts.slice(0, 12).map(post => `
    <div class="card">
      <h3>${post.title}</h3>
      <p>${post.body}</p>
      <p class="meta">Post ID: ${post.id}  |  User ID: ${post.userId}</p>
    </div>
  `).join("");
}

// Step 4: Render users into cards
function renderUsers(users) {
  output.innerHTML = users.map(user => `
    <div class="card">
      <h3>${user.name}</h3>
      <p>📧 ${user.email}</p>
      <p>📞 ${user.phone}</p>
      <p>🌐 ${user.website}</p>
      <p class="meta">${user.address.city}, ${user.address.zipcode}</p>
    </div>
  `).join("");
}

function setLoading(msg) {
  status.textContent = msg;
  btnPosts.disabled = true;
  btnUsers.disabled = true;
}

function setDone(msg) {
  status.textContent = msg;
  btnPosts.disabled = false;
  btnUsers.disabled = false;
}

// Fetch Posts button
btnPosts.addEventListener("click", () => {
  setLoading("Fetching posts...");
  output.innerHTML = "";
```

```
  ajaxGet(
    "https://jsonplaceholder.typicode.com/posts",
    (data) => {
      renderPosts(data);
      setDone(`Showing ${Math.min(data.length, 12)} of ${data.length} posts`);
    },
    (err) => {
      output.innerHTML = `<p class="error">${err}</p>`;
      setDone("");
    }
  );
});

// Fetch Users button
btnUsers.addEventListener("click", () => {
  setLoading("Fetching users...");
  output.innerHTML = "";

  ajaxGet(
    "https://jsonplaceholder.typicode.com/users",
    (data) => {
      renderUsers(data);
      setDone(`Showing ${data.length} users`);
    },
    (err) => {
      output.innerHTML = `<p class="error">${err}</p>`;
      setDone("");
    }
  );
});

// Clear button
btnClear.addEventListener("click", () => {
  output.innerHTML = "";
  status.textContent = "";
});
```

Step 4: Deploying using Live Server

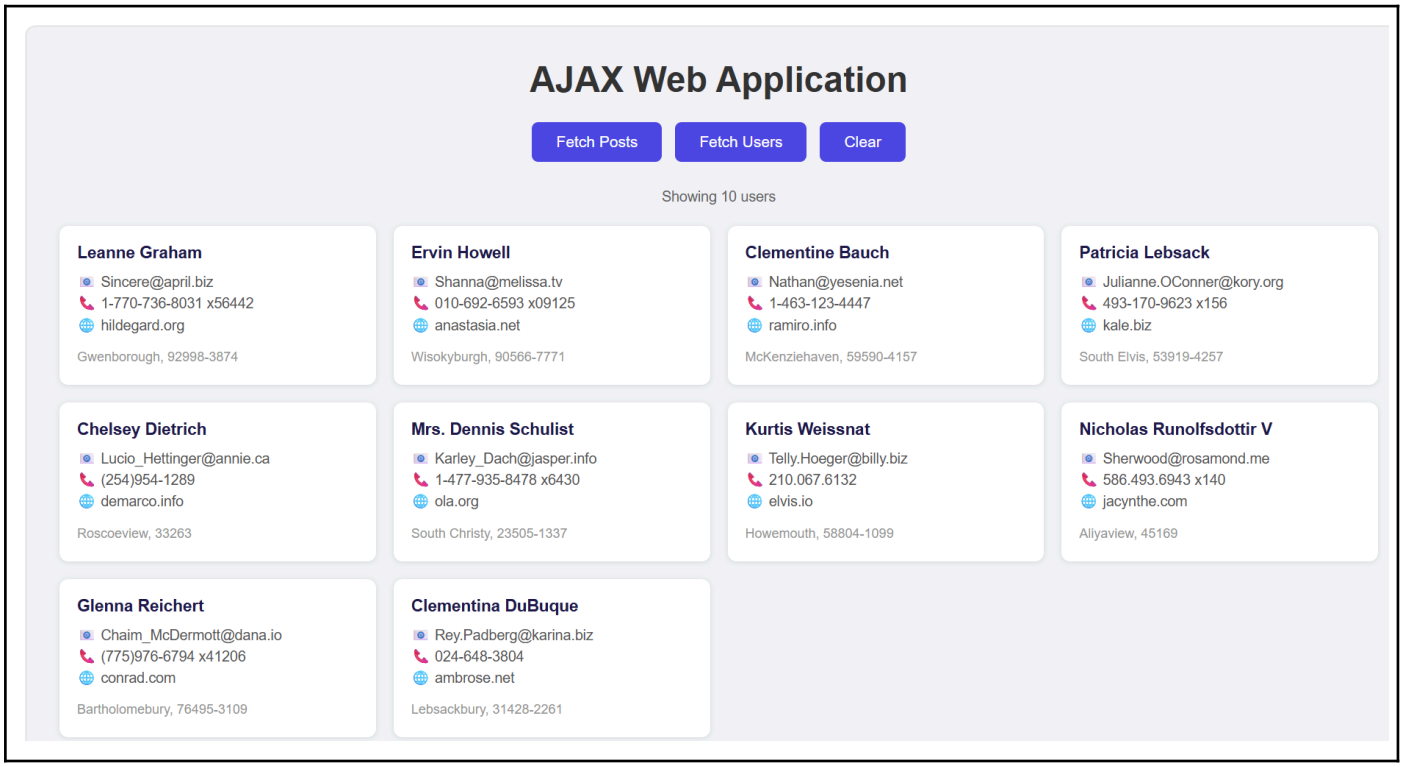
Post Explorer (AJAX App) at http://127.0.0.1:5500/WEBX-10/index.html

The application was executed using a web browser with the help of a local server such as Live Server. This ensured proper functioning of AJAX requests.



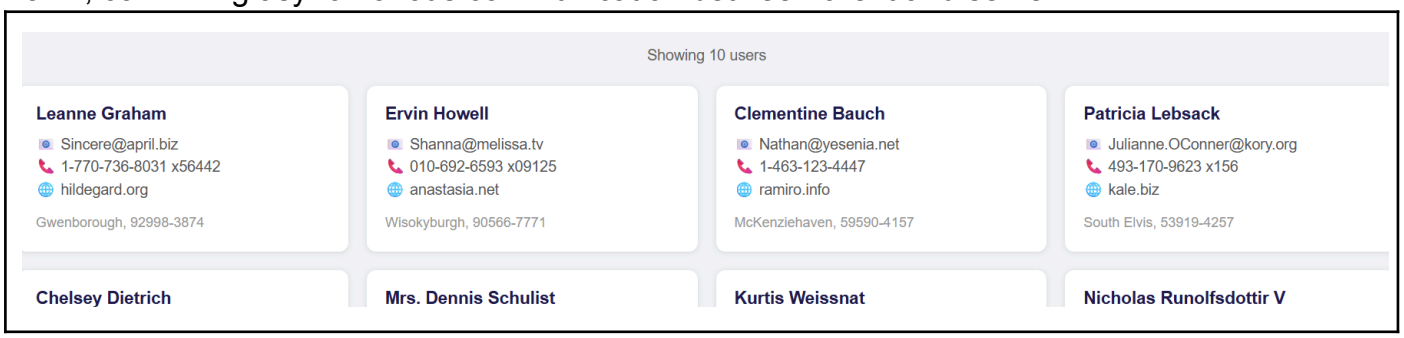
Step 5: Displaying Data Dynamically

The extracted data was dynamically displayed on the webpage by creating HTML elements using JavaScript. Each post included a title, summary, and a link to read more details.



Step 9: Result Observation

It was observed that the application successfully fetched and displayed data dynamically using AJAX, confirming asynchronous communication between client and server.



CONCLUSION:-

This experiment demonstrates the practical implementation of AJAX for building dynamic and responsive web applications. By enabling asynchronous communication, it allows data to be fetched from the server without interrupting the user experience. The use of JSON as a data format simplifies data handling and integration with JavaScript. Through this process, we learned how to send HTTP requests, process server responses, and update the webpage content dynamically. This approach significantly enhances performance and efficiency compared to traditional page reload methods. Overall, AJAX plays a crucial role in modern web development by making applications faster, smoother, and more interactive.